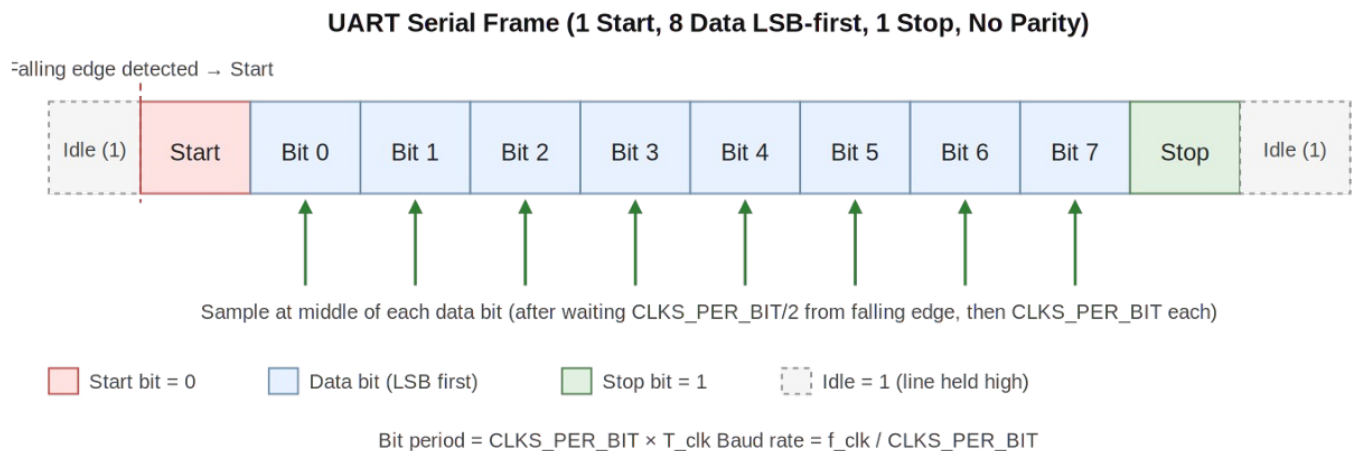


UART — Universal Asynchronous Receiver / Transmitter

A UART (Universal Asynchronous Receiver / Transmitter) is a serial communication interface that exchanges bytes over a single data line per direction with no shared clock between transmitter and receiver. Because the link is asynchronous, both ends must agree in advance on the bit timing (the baud rate); the receiver then locks onto each incoming frame by detecting the falling edge of the start bit and sampling each data bit at the middle of its bit period.

This design implements a UART transmitter (`uart_tx`) and a UART receiver (`uart_rx`) as two independent Verilog modules with a single shared parameter, `CLKS_PER_BIT`, that sets the number of clock cycles per bit. The frame format is fixed at 1 start bit, 8 data bits (LSB first), 1 stop bit, and no parity. The transmitter additionally exposes a `Tx_Active` signal that can be used to drive a tri-state buffer for half-duplex operation; the two modules together also support full-duplex operation on separate pins.



Module Overview

The design is partitioned into two independent modules:

- `uart_tx` — accepts an 8-bit byte in parallel and shifts it out serially as a UART frame.
- `uart_rx` — watches the serial line, detects start bits, samples 8 data bits at the middle of each bit cell, and presents the result as an 8-bit byte with a one-cycle data-valid strobe.

Both modules share a single integer parameter, `CLKS_PER_BIT`, that converts the system clock frequency into a baud rate. No shared signals are required between the two modules — they communicate only through the serial line.

Parameters

Parameter	Function
CLKS_PER_BIT	Integer that sets the number of <code>clk</code> cycles in one UART bit period. Computed as (frequency of <code>clk</code>) / (baud rate). Example: a 10 MHz clock targeting 115,200 baud gives $10,000,000 / 115,200 \approx 87$. The same value must be used by both the transmitter and the receiver. Must be ≥ 2 for the half-bit sample point in the receiver to be meaningful.

uart_rx — Data Ports

Signal Name	Direction	Function
<code>clk</code>	input	System clock. All receiver logic is synchronous to its rising edge.
<code>Rx_serial</code>	input	Asynchronous serial data input. Idle = 1. A high-to-low transition on this pin is interpreted as the start of a new frame. This input is double-flopped internally (<code>Rx_serial_FF1</code> , <code>Rx_serial_FF2</code>) to mitigate metastability before being used by the state machine.
<code>Rx_DV</code>	output	Receive Data Valid. Driven high for exactly one <code>clk</code> cycle when a complete byte has been received and is available on <code>Rx_Byte</code> . Otherwise 0.
<code>Rx_Byte</code>	output [7:0]	8-bit parallel output holding the most recently received byte. Bit 0 is the first data bit received from the line (LSB first). Valid when <code>Rx_DV</code> = 1 and holds its value until the next byte completes.

uart_tx — Data Ports

Signal Name	Direction	Function
<code>clk</code>	input	System clock. All transmitter logic is synchronous to its rising edge.
<code>Tx_DV</code>	input	Transmit Data Valid. Pulsed high (typically for one clock) by the user logic to request transmission of <code>Tx_Byte</code> . The byte is latched into <code>Tx_Latched_Byte</code> on the cycle the IDLE state sees <code>Tx_DV</code> = 1.
<code>Tx_Byte</code>	input [7:0]	8-bit parallel data to transmit. Must be valid on the cycle <code>Tx_DV</code> is sampled. Bit 0 is sent first on the serial line (LSB first).
<code>Tx_Active</code>	output	High for the entire duration of a frame (start bit through stop bit). Intended to enable a tri-state buffer driving the shared serial line in half-duplex configurations.
<code>Tx_Serial</code>	output	Serial data output. Held high when idle. Drives start bit (0), the 8 data bits LSB first, then stop bit (1).
<code>Tx_Done</code>	output	Driven high for one <code>clk</code> cycle when transmission of the current byte has fully completed (after the stop bit). Otherwise 0.

uart_rx — State Machine

The receiver is a 5-state Moore machine (STATE_REG) that runs entirely on the rising edge of `clk`. A free-running clock counter (CLK_COUNTER, range 0 to CLKS_PER_BIT - 1) measures the bit period, and a 3-bit bit-index counter (BIT_IDX_COUNTER) selects which bit of Rx_Acc_Byte is written.

State	Behavior
IDLE	Hold Rx_DV = 0; clear BIT_IDX_COUNTER and CLK_COUNTER. Wait for the synchronized serial input (Rx_serial_FF2) to go low, indicating a start bit, then advance to RX_START_BIT.
RX_START_BIT	Wait $(\text{CLKS_PER_BIT} - 1) / 2$ clocks to reach the middle of the start bit, then re-check the line. If still low it is confirmed as a valid start bit and the state advances to RX_DATA_BITS with CLK_COUNTER reset. If the line is high it was a glitch; return to IDLE.
RX_DATA_BITS	Wait CLKS_PER_BIT - 1 clocks per bit, then sample the line into Rx_Acc_Byte[BIT_IDX_COUNTER]. Increment BIT_IDX_COUNTER until all 8 bits are captured, then advance to RX_STOP_BIT.
RX_STOP_BIT	Wait CLKS_PER_BIT - 1 clocks for the stop bit to finish, then pulse Rx_DV high and advance to CLEANUP.
CLEANUP	Spend one clock cycle deasserting Rx_DV and then return to IDLE. This guarantees Rx_DV is a single-cycle pulse.

uart_tx — State Machine

The transmitter is a 5-state Moore machine (STATE_REG) that runs on the rising edge of `clk`. The same CLK_COUNTER / BIT_IDX_COUNTER pair is used, but it drives the output line rather than sampling it. The byte to send is latched (into Tx_Latched_Byte) on entry to TX_START_BIT so that the user is free to change Tx_Byte once the transmission has started.

State	Behavior
IDLE	Drive Tx_Serial = 1, Tx_Active = 0, Tx_Done = 0; clear counters. When Tx_DV = 1, latch Tx_Byte into Tx_Latched_Byte and advance to TX_START_BIT.
TX_START_BIT	Drive Tx_Serial = 0 and Tx_Active = 1. Hold for CLKS_PER_BIT clocks, then advance to TX_DATA_BITS.
TX_DATA_BITS	Drive Tx_Serial = Tx_Latched_Byte[BIT_IDX_COUNTER]. Hold for CLKS_PER_BIT clocks per bit, increment BIT_IDX_COUNTER until all 8 bits have been transmitted, then advance to TX_STOP_BIT.
TX_STOP_BIT	Drive Tx_Serial = 1 for CLKS_PER_BIT clocks. On exit, pulse Tx_Done = 1 and advance to CLEANUP.
CLEANUP	Spend one clock cycle deasserting Tx_Active and Tx_Done, then return to IDLE.

General Notes

1. Frame format is fixed: 1 start bit, 8 data bits, 1 stop bit, no parity. The least significant bit of the data byte is transmitted first.
2. The line idles high. The start bit is a logic 0 held for one full bit period; the stop bit is a logic 1 held for one full bit period.
3. Both `uart_tx` and `uart_rx` must be configured with the same `CLKS_PER_BIT`. Tolerable mismatch is roughly $\pm 2\%$ before the sample point at the end of bit 7 walks outside its bit cell.
4. `uart_rx` double-flops `Rx_serial` through `Rx_serial_FF1` / `Rx_serial_FF2` before using it. This is required because the input is asynchronous to `clk` and a single-flop sample can become metastable.
5. The receiver locks the bit cell by waiting $(\text{CLKS_PER_BIT} - 1) / 2$ cycles into the start bit and re-sampling. From that point on it waits one full bit period between samples so that all eight data bits are sampled near the middle of their cells.
6. `uart_tx` exposes `Tx_Active` so that an external tri-state buffer can drive a shared half-duplex line only while a frame is being sent. For full-duplex operation `Tx_Active` can simply be left unconnected.
7. `Rx_DV` and `Tx_Done` are strobes — they are high for exactly one `clk` cycle. Downstream logic must register the data on the cycle the strobe is high.
8. `uart_rx` does not check the stop bit value; it only waits for the stop bit period to elapse before asserting `Rx_DV`. A framing-error check (verifying the line is high during stop) is a straightforward extension.
9. The design does not include parity generation or checking, hardware flow control (RTS/CTS), or a FIFO. These are intentionally left to the surrounding system.
10. `uart_tx` ignores `Tx_DV` pulses that arrive while it is not in `IDLE` — that is, while a previous frame is still being shifted out. User logic should gate new requests on `Tx_Active` being low (or wait for `Tx_Done`).

Design Checklist (Verilog)

1. Declare `CLKS_PER_BIT` as a module parameter on both `uart_rx` and `uart_tx`; do not hard-code the baud rate inside the modules.
2. Use named local parameters for the five FSM states in each module to keep waveforms readable (`IDLE`, `RX_START_BIT`/`TX_START_BIT`, etc.).
3. Double-register `Rx_serial` through `Rx_serial_FF1`/`Rx_serial_FF2` before using it anywhere in the receiver FSM.
4. Size `CLK_COUNTER` to hold $\text{CLKS_PER_BIT} - 1$ and `BIT_IDX_COUNTER` to 3 bits (range 0..7).
5. Initialize `Tx_Serial`, `Tx_Active`, `Tx_Done`, and `Rx_DV` to safe idle values (1 for the serial line, 0 for the strobes) so the design behaves correctly out of reset, even without an explicit reset input.
6. Make sure `Rx_DV` and `Tx_Done` are exactly one-cycle pulses by clearing them in `CLEANUP` / `IDLE`.
7. Verify in simulation that a TX-out $\rightarrow$ RX-in loopback reproduces the original byte, and that back-to-back frames are accepted without dropping bytes.
8. Verify that asserting `Tx_DV` mid-frame is ignored, and that the receiver returns to `IDLE` on a glitched (one-clock-wide) low pulse on `Rx_serial`.